Next.js — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Setup

02 App Router and File-based Routing

03 Server and Client Components

04 Links, Navigation and Layouts

05 Styling

06 Images and Metadata

Intermediate (6 topics)

07 Data Fetching in Server Components

08 Dynamic Routes and generateStaticParams

09 Route Handlers (API)

10 Loading, Error and Not-found UI

11 Forms and Server Actions

12 Environment Variables and Config

Advanced (6 topics)

13 Rendering Strategies: SSG, SSR, ISR

14 Caching and Revalidation

15 Authentication

16 Middleware

17 Database Integration

18 Streaming and Suspense

Master (4 topics)

19	Performance Optimisation
20	Testing and CI
21	Real Project: Full-stack Blog or Store
22	Deployment and Self-hosting

Beginner Level

01. Introduction and Setup

DEFINITION

Next.js is a React framework with routing, server rendering, API routes and optimisation built in.

SYNTAX

```
npx create-next-app@latest
```

EXAMPLE

```
npx create-next-app@latest my-app
cd my-app && npm run dev
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A ready-made shop building with plumbing, wiring and signage already installed; Next.js gives React apps routing, rendering and optimisation out of the box.

தமிழ்

குழாய், மின்சாரம், பலகை எல்லாம் ஏற்கனவே அமைக்கப்பட்ட கடைக் கட்டிடம் போல Next.js React செயலிகளுக்கு routing, rendering, மேம்பாடுகளை தருகிறது.

PROS

- SEO friendly (server rendering)
- Built-in routing, image and font optimisation
- Full-stack in one project

CONS

- Opinionated, frequent changes
- Hosting features tied to platform
- Learning curve of caching

ADVANCED / NEW FEATURES

- Next.js 16: Turbopack as default bundler, Cache Components (`use cache`), React 19 support, `proxy.ts` replacing middleware

02. App Router and File-based Routing

DEFINITION

Folders inside `app/` become URLs; `page.tsx` is the page, `layout.tsx` wraps pages.

SYNTAX

```
app/folder/page.tsx -> /folder
```

EXAMPLE

```
app/
  layout.tsx
  page.tsx          -> /
  about/page.tsx    -> /about
  blog/[slug]/page.tsx -> /blog/hello
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Platform numbers at a bus stand: each folder name becomes a URL path automatically.

தமிழ்

பேருந்து நிலைய நடைமேடை எண்கள் போல ஒவ்வொரு folder பெயரும் தானாக URL ஆகிறது.

PROS

- No router config
- Nested layouts
- Co-located files

CONS

- Special file names to remember
- Moving files changes URLs

ADVANCED / NEW FEATURES

- Parallel and intercepting routes
- Typed routes

03. Server and Client Components

DEFINITION

Components are **Server Components** by default (no client JS). Add `'use client'` for state, effects and event handlers.

SYNTAX

```
'use client';
```

EXAMPLE

```
'use client';
import { useState } from 'react';
export default function Counter() { const [n, setN] = useState(0); return <button onClick={()
=> setN(n + 1)}>{n}</button>; }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

In a restaurant, the kitchen (server) prepares food and only the serving counter (client) handles customer interaction; Server Components do work on the server, Client Components handle clicks.

தமிழ்

உணவகத்தில் சமையலறை (server) உணவு தயாரிக்கிறது, பரிமாறும் கவுண்டர் (client) வாடிக்கையாளர் தொடர்பைக் கையாளுகிறது; Server Components server-இல் வேலை செய்கின்றன.

PROS

- Less JavaScript sent to browser
- Direct data access on server
- Better performance

CONS

- Mental model is new
- Some libraries need 'use client'

ADVANCED / NEW FEATURES

- React Server Components stable in App Router
- `use cache` directive

04. Links, Navigation and Layouts

DEFINITION

Use `<Link>` for client-side navigation and nested layouts for shared UI; `useRouter` for programmatic navigation.

SYNTAX

```
<Link href="/about">About</Link>
```

EXAMPLE

```
import Link from 'next/link';
<Link href="/about">About</Link>
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Moving between rooms inside a mall without going out to the street; `Link` navigates without a full reload and prefetches pages.

தமிழ்

வணிக வளாகத்தில் தெருவுக்குச் செல்லாமல் அறைகளுக்கு இடையே செல்வது போல `Link` முழுப் பக்க ஏற்றம் இல்லாமல் செல்கிறது.

PROS

- Instant navigation
- Automatic prefetching

CONS

- Prefetch can waste data on slow networks

ADVANCED / NEW FEATURES

- Layout deduplication and incremental prefetching (16)

05. Styling

DEFINITION

CSS Modules, global CSS, Tailwind CSS and CSS-in-JS options; fonts via `next/font`.

SYNTAX

```
import styles from './x.module.css';
```

EXAMPLE

```
import styles from './page.module.css';
<h1 className={styles.title}>Hi</h1>
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Each shop in a mall paints its own board without affecting neighbours; CSS Modules scope styles locally.

தமிழ்

வளாகத்தில் ஒவ்வொரு கடையும் தன் பலகையை மற்றவர்களைப் பாதிக்காமல் வண்ணம் தீட்டுவது போல CSS Modules scoped style தருகின்றன.

PROS

- Scoped CSS
- Tailwind integration
- Font optimisation

CONS

- CSS-in-JS needs client components

ADVANCED / NEW FEATURES

- Tailwind CSS v4
- `next/font` self-hosting

06. Images and Metadata

DEFINITION

`next/image` serves optimised images; export `metadata` for SEO titles and descriptions.

SYNTAX

```
export const metadata = { title: '...' };
```

EXAMPLE

```
export const metadata = { title: 'My Site', description: 'Learn Next.js' };
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop name board and a Google Maps listing so customers can find you; metadata and optimised images help search and speed.

தமிழ்

கடைப் பலகையும் மேப் பட்டியலும் வாடிக்கையாளர் உங்களைக் கண்டுபிடிக்க உதவுவது போல metadata, optimised images தேடலுக்கும் வேகத்துக்கும் உதவுகின்றன.

PROS

- Better SEO
- Automatic image resizing
- Prevents layout shift

CONS

- Remote images need domain config

ADVANCED / NEW FEATURES

- `generateMetadata`
- Open Graph image generation

Intermediate Level

07. Data Fetching in Server Components

DEFINITION

Server Components can be `async` and fetch data directly; control caching with `fetch` options and `revalidate`.

SYNTAX

```
const res = await fetch(url, { next: { revalidate: 60 } });
```

EXAMPLE

```
export default async function Page() {
  const res = await fetch('https://api.example.com/posts', { next: { revalidate: 60 } });
  const posts = await res.json();
  return <ul>{posts.map((p: any) => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bakery fixes fresh stock every hour instead of baking per customer; server fetching with revalidation caches results.

தமிழ்

பேக்கரி ஒவ்வொரு வாடிக்கையாளருக்கும் தனியாக சுடாமல் மணிக்கொருமுறை புதிய இருப்பு வைப்பது போல revalidation முடிவுகளைக் கேச் செய்கிறது.

PROS

- Fetch next to the component
- No client waterfalls
- Built-in caching

CONS

- Caching rules can surprise you

ADVANCED / NEW FEATURES

- Next 15 changed fetch default to uncached
- Cache Components (`use cache`) in 16

08. Dynamic Routes and generateStaticParams

DEFINITION

Dynamic segments `[id]` receive params; `generateStaticParams` pre-renders pages at build time.

SYNTAX

```
export async function generateStaticParams() { }
```

EXAMPLE

```
export async function generateStaticParams() { return [{ slug: 'a' }, { slug: 'b' }]; }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Printing posters for known products in advance so customers get them instantly; dynamic routes pre-render known pages.

தமிழ்

தெரிந்த பொருட்களுக்கான சுவரொட்டிகளை முன்கூட்டியே அச்சிடுவது போல dynamic routes அறியப்பட்ட பக்கங்களை முன்பே உருவாக்குகின்றன.

PROS

- Fast static pages
- Fallback for new paths

CONS

- Build time grows with pages

ADVANCED / NEW FEATURES

- `params` is async in Next 15+ (`await params`)

09. Route Handlers (API)

DEFINITION

Create API endpoints with `route.ts` exporting HTTP-method functions.

SYNTAX

```
export async function GET() { return Response.json({}); }
```

EXAMPLE

```
// app/api/hello/route.ts
export async function GET() { return Response.json({ message: 'Hello' }); }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A small service counter at the back of the shop answering data requests; route handlers are API endpoints in the same project.

தமிழ்

கடையின் பின்புறம் தரவுக் கோரிக்கைகளுக்குப் பதில் சொல்லும் சிறிய சேவை கவுண்டர் போல route handlers அதே project-இல் API endpoints ஆகும்.

PROS

- No separate backend for small needs
- Web-standard Request/Response

CONS

- Not ideal for heavy backends
- Cold starts on serverless

ADVANCED / NEW FEATURES

- Uses standard Web APIs
- Typed route handlers

10. Loading, Error and Not-found UI

DEFINITION

Special files `loading.tsx`, `error.tsx`, `not-found.tsx` provide streaming fallbacks and error boundaries per route.

SYNTAX

```
loading.tsx / error.tsx / not-found.tsx
```

EXAMPLE

```tsx
// app/blog/loading.tsx
export default function Loading() { return <p>Loading posts...</p>; }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A 'please wait' board while food is prepared, and a 'sorry' board if an item is unavailable; special files show loading and error screens.

தமிழ்

உணவு தயாராகும் வரை 'காத்திருக்கவும்' பலகை, இல்லையென்றால் 'மன்னிக்கவும்' பலகை போல loading, error கோப்புகள் திரைகளைக் காட்டுகின்றன.

PROS

- Automatic Suspense boundaries
- Per-route error isolation

CONS

- Easy to forget skeleton sizes

ADVANCED / NEW FEATURES

- Streaming with `loading.tsx`
- `unstable_rethrow`

11. Forms and Server Actions

DEFINITION

Server Actions are async functions marked `'use server'` that run on the server and can be used as form actions.

SYNTAX

```
async function action(formData) { 'use server'; }
```

EXAMPLE

```
async function create(formData: FormData) {
  'use server';
  await db.post.create({ data: { title: formData.get('title') as string } });
}
<form action={create}>...</form>
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Dropping a complaint slip into a box that goes straight to the manager; a Server Action sends form data directly to server code.

தமிழ்

புகார் சீட்டை நேரடியாக மேலாளரிடம் செல்லும் பெட்டியில் போடுவது போல Server Action படிவத் தரவை நேரடியாக server குறியீட்டுக்கு அனுப்புகிறது.

PROS

- No manual API route
- Progressive enhancement
- Type-safe

CONS

- Must validate and authorise yourself
- Hidden endpoints

ADVANCED / NEW FEATURES

- Stable since Next 14
- `after()` for post-response work

12. Environment Variables and Config

DEFINITION

`.env.local` for secrets; only `NEXT_PUBLIC_` variables reach the browser. Configure in `next.config.ts`.

SYNTAX

```
.env.local -> NEXT_PUBLIC_*
```

EXAMPLE

```
DATABASE_URL=...
NEXT_PUBLIC_API=https://api.example.com
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop safe key is kept private but the shop name board is public; only `NEXT_PUBLIC_` variables go to the browser.

தமிழ்

கடைப் பாதுகாப்புப் பெட்டிச் சாவி மறைவாக இருக்கும், பலகை பொதுவில் இருக்கும்; `NEXT_PUBLIC_` மாறிகள் மட்டுமே browser-க்கு செல்லும்.

PROS

- Secrets stay on the server
- Per-environment config

CONS

- Easy to leak by wrong prefix

ADVANCED / NEW FEATURES

- Typed env helpers via libraries

Advanced Level

13. Rendering Strategies: SSG, SSR, ISR

DEFINITION

Static generation (build), server rendering (each request), incremental regeneration (`revalidate`), and Partial Prerendering; pick per page.

SYNTAX

```
export const revalidate = 3600;
export const dynamic = 'force-dynamic';
```

EXAMPLE

```
export const dynamic = 'force-dynamic'; // SSR
export const revalidate = 3600;          // ISR
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Newspaper models: printed edition (static), live TV (per request), and a morning edition refreshed hourly (ISR).

தமிழ்

நாளிதழ் அச்சு (static), நேரலை தொலைக்காட்சி (SSR), மணிக்கொருமுறை புதுப்பிக்கப்படும் பதிப்பு (ISR) போல rendering வகைகள்.

PROS

- Choose speed or freshness per page
- Great performance

CONS

- Wrong choice causes stale or slow pages

ADVANCED / NEW FEATURES

- Partial Prerendering / Cache Components combine static and dynamic

14. Caching and Revalidation

DEFINITION

Understand request memoisation, Data Cache and Router Cache; use `revalidatePath` / `revalidateTag` after mutations.

SYNTAX

```
revalidatePath('/blog');
revalidateTag('posts');
```

EXAMPLE

```
import { revalidatePath } from 'next/cache';
revalidatePath('/blog');
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

When prices change, only the changed notice boards are replaced, not the entire shop; revalidation refreshes selected cached data.

தமிழ்

விலை மாறினால் முழுக் கடையையும் மாற்றாமல் மாறிய பலகைகள் மட்டும் மாற்றப்படுவது போல revalidation தேர்ந்தெடுத்த கேச் தரவைப் புதுப்பிக்கும்.

PROS

- Fine-grained cache control
- Fast pages with fresh data

CONS

- Multiple cache layers are confusing

ADVANCED / NEW FEATURES

- `updateTag` and refined caching APIs in v16

15. Authentication

DEFINITION

Use Auth.js (NextAuth), sessions in cookies, protect routes with middleware and verify on the server.

SYNTAX

```
export { auth as middleware } from '@/auth';
```

EXAMPLE

```
export { auth as middleware } from '@/auth';
export const config = { matcher: ['/dashboard/:path*'] };
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An entry gate where your pass is checked and your identity recorded; auth sessions protect pages and APIs.

தமிழ்

நுழைவாயிலில் உங்கள் அனுமதிச் சீட்டு சோதிக்கப்பட்டு அடையாளம் பதிவாவது போல auth sessions பக்கங்களையும் API-களையும் பாதுகாக்கின்றன.

PROS

- Mature libraries (Auth.js, Clerk)
- Cookie-based sessions

CONS

- Must verify on the server too

ADVANCED / NEW FEATURES

- Auth.js continuing as part of Better Auth ecosystem
- Passkeys/WebAuthn support

16. Middleware

DEFINITION

Code that runs before a request completes: redirects, rewrites, auth checks, A/B tests, headers.

SYNTAX

```
export function middleware(req) { }
```

EXAMPLE

```
export function middleware(req: NextRequest) {
  if (!req.cookies.get('token')) return NextResponse.redirect(new URL('/login', req.url));
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A security guard at the main gate who redirects visitors before they reach the building; middleware runs before a request completes.

தமிழ்

முதன்மை வாயிலில் காவலர் கட்டிடத்துக்கு முன்பே பார்வையாளர்களைத் திருப்புவது போல middleware கோரிக்கை முடியும் முன் இயங்கும்.

PROS

- Fast redirects/rewrites
- Runs at the edge

CONS

- Limited runtime APIs

- Hard to debug

ADVANCED / NEW FEATURES

- Renamed to `proxy.ts` in Next.js 16
- Node.js runtime for proxy

17. Database Integration

DEFINITION

Use Prisma or Drizzle with PostgreSQL/MySQL inside server components, actions and route handlers.

SYNTAX

```
const rows = await prisma.user.findMany();
```

EXAMPLE

```
const users = await prisma.user.findMany({ where: { active: true } });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop's ledger book kept safely in the back office; the app talks to a database from server code.

தமிழ்

கடையின் கணக்குப் புத்தகம் பின்புற அலுவலகத்தில் பாதுகாப்பாக இருப்பது போல செயலி server குறியீட்டிலிருந்து database-ஐ அணுகுகிறது.

PROS

- Type-safe queries
- Direct access from server components

CONS

- Connection limits on serverless
- Never expose DB to client

ADVANCED / NEW FEATURES

- Prisma 6/7 and Drizzle improvements
- Vercel Postgres/Neon integrations

18. Streaming and Suspense

DEFINITION

Wrap slow parts in `<Suspense>` so the rest of the page streams to the browser immediately.

SYNTAX

```
<Suspense fallback={<Skeleton />}><Slow /></Suspense>
```

EXAMPLE

```
<Suspense fallback={<Skeleton />}><Reviews /></Suspense>
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

At a restaurant, starters arrive immediately while the main course is still cooking; streaming sends ready parts first.

தமிழ்

உணவகத்தில் முதன்மை உணவு தயாராகும் வரை ஸ்டார்டர்கள் உடனே வருவது போல streaming தயாரான பகுதிகளை முதலில் அனுப்புகிறது.

PROS

- Faster first content
- Better perceived speed

CONS

- Needs skeleton design
- Layout shifts if careless

ADVANCED / NEW FEATURES

- Partial Prerendering
- Streaming metadata

Master Level

19. Performance Optimisation

DEFINITION

Optimise images/fonts/scripts, reduce client JS with Server Components, analyse bundles, use Core Web Vitals and caching.

SYNTAX

```
ANALYZE=true npm run build
```

EXAMPLE

```
npm i @next/bundle-analyzer
ANALYZE=true npm run build
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A traffic police studying jams to open a new flyover; analyzing bundles shows what slows pages down.

தமிழ்

நெரிசலை ஆய்வு செய்து மேம்பாலம் அமைப்பது போல bundle பகுப்பாய்வு பக்கத்தை எது மெதுவாக்குகிறது என்பதைக் காட்டும்.

PROS

- Real measurable gains
- Core Web Vitals improvement

CONS

- Measuring takes effort

ADVANCED / NEW FEATURES

- Turbopack fast builds
- React Compiler support

20. Testing and CI

DEFINITION

Unit tests with Vitest/Jest, component tests with Testing Library, end-to-end with Playwright, GitHub Actions pipeline.

SYNTAX

```
npx playwright test
```

EXAMPLE

```
npx playwright test
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Quality inspectors test each item before the shop opens; unit and end-to-end tests check the app.

தமிழ்

கடை திறக்கும் முன் தர ஆய்வாளர்கள் ஒவ்வொரு பொருளையும் சோதிப்பது போல unit மற்றும் end-to-end tests செயலியைச் சோதிக்கின்றன.

PROS

- Confidence in releases
- Automated CI checks

CONS

- Async Server Components are hard to unit test
- Flaky e2e tests

ADVANCED / NEW FEATURES

- Experimental Vitest/Playwright guidance for App Router

21. Real Project: Full-stack Blog or Store

DEFINITION

Auth, database, CRUD with Server Actions, image upload, SEO metadata, ISR and deployment on Vercel.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A complete online store with login, products and orders: the typical full-stack practice project.

தமிழ்

உள்நுழைவு, பொருட்கள், ஆர்டர்கள் உடன் முழுமையான ஆன்லைன் கடை: வழக்கமான full-stack பயிற்சி project.

PROS

- Covers routing, data, auth, deployment

CONS

- Takes time

ADVANCED / NEW FEATURES

- Use latest App Router, Server Actions and Cache Components

22. Deployment and Self-hosting

DEFINITION

Deploy to Vercel, or self-host with `next start` / Docker `output: 'standalone'`; configure env vars and caching.

SYNTAX

```
npm run build && npm start
```

EXAMPLE

```
npm run build && npm start
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Opening the shop to the public after final preparation; deployment publishes the app for users.

தமிழ்

இறுதித் தயாரிப்புக்குப் பின் கடையை மக்களுக்குத் திறப்பது போல deployment செயலியைப் பயனர்களுக்கு வெளியிடுகிறது.

PROS

- One-click on Vercel
- Self-host with Docker/Node
- Static export possible

CONS

- Some features need platform support when self-hosting

ADVANCED / NEW FEATURES

- `output: 'standalone'`
- Adapters API (stable in v16)

Interview Question Bank

Q1. Server vs Client Components?
Server Components render on the server and ship no JS; Client Components (`'use client'`) hydrate in the browser and use state/effects.

Q2. SSR vs SSG vs ISR?
SSR renders per request; SSG renders at build; ISR regenerates static pages in the background after a time or event.

Q3. What is the App Router?
The `app/` directory routing system with layouts, server components, streaming and server actions.

Q4. What are Server Actions?
Server-side functions invoked from forms or client code without writing an API route.

Q5. What is middleware used for?
Running logic before requests: auth redirects, rewrites, localisation, headers.

Q6. How does `next/image` help?
Automatic resizing, lazy loading, modern formats and layout shift prevention.

Q7. How do you handle SEO?
`metadata` / `generateMetadata`, server rendering, sitemaps, canonical URLs and structured data.

Q8. What is hydration?
React attaching event handlers to server-rendered HTML in the browser.

Q9. How are environment variables exposed?
Only variables prefixed `NEXT_PUBLIC_` are bundled for the browser; others stay server-side.

Q10. Pages Router vs App Router?
Pages Router uses `pages/` with `getServerSideProps`; App Router is the newer, component-based model.

Capstone Project Ideas

1. Personal blog with MDX
2. E-commerce store with Stripe
3. Dashboard with auth and Postgres
4. URL shortener
5. Job board with search